UNIDAD 4: PROYECTO

OPTIMIZACION Y DOCUMENTACION.

Proyecto realizado por: Dennis Soleto Yunga

INDICE

REFACTORIZACION (1) (PAG 03-09):

Conceptos básicos:

- ¿Qué es la refactorización?
- Objetivo principal.
- Beneficios de la refactorización.

Limitaciones:

- No añade funcionalidades nuevas.
- Riesgos si no existen pruebas.
- Tiempo y costos.
- No se corrigen errores de diseño profundo.

Patrones de refactorización mas usuales:

- Renombrar variables/métodos.
- Extraer método.
- Mover método o clase.
- Eliminar código duplicado.
- Simplificación de condicionales.

Refactorización y pruebas:

- Importancia de las pruebas.
- Evitar que el comportamiento cambie.
- Pruebas unitarias como apoyo.

Herramientas de ayuda:

- Funciones automáticas del IDE (4).

CONTROL DE VERSIONES (2) (PAG 10-16):

Desarrollo colectivo:

- Trabajo en equipo sobre el mismo proyecto.
- Control de cambios.
- Seguimiento de autores y modificaciones.

Herramientas de control:

- Git.
- SVN.
- Mercurial.
- Utilidad en proyectos reales.

Características principales:

- Historial de versiones.
- Trabajo en ramas (branches).
- Fusiones (merge).
- Resolución de conflictos.

Estructuras:

- Modelo cliente/servidor.
- Repositorio remoto.
- Copias locales.

Operaciones con clientes:

- Descarga, Modificaciones y Actualizaciones.
- Subida de cambios, comparación y restauración.
- Resolución de conflictos y consulta de historial.

DOCUMENTACION DE SOFTWARE (3) (PAG 17-19):

Uso de comentarios:

- Finalidad de comentarios.
- Buenas practicas.
- Evitar comentarios innecesarios.

Herramientas de documentación:

- Generación de documentación desde el código.

Alternativas de documentación:

- Wikis de proyecto.
- README.
- Manuales técnicos.
- Diagramas de UML.

CONCLUSION (4) (PAG 20):

- Resumen de ideas principales y punto de vista respecto al proyecto realizado.
- Fin del proyecto.

REFACTORIZACION

CONCEPTOS BÁSICOS:

¿QUÉ ES LA REFACTORIZACIÓN?:

La refactorización es una práctica del desarrollo de software que consiste en modificar el código fuente de un programa con el objetivo de mejorar su estructura interna sin alterar su funcionamiento externo.

Es decir, el programa sigue haciendo exactamente lo mismo, pero el código queda mejor organizado, más claro y más fácil de entender. Esta técnica se utiliza habitualmente durante el mantenimiento del software para evitar que el código se vuelva complejo o defectuoso (code smells), y difícil de manejar con el paso del tiempo.

OBJETIVO PRINCIPAL DE LA REFACTORIZACIÓN:

El objetivo principal de la refactorización es mejorar la calidad del código sin cambiar su comportamiento.

Esto significa que, después de refactorizar, el programa debe producir los mismos resultados que antes. La idea es facilitar futuras modificaciones, reducir la complejidad y permitir que otros desarrolladores comprendan el código con mayor rapidez, evitando errores y malentendidos.

BENEFICIOS DE LA REFACTORIZACIÓN:

La refactorización aporta múltiples beneficios al desarrollo de software, como:

- Mejorar la legibilidad del código, lo que permite entenderlo más fácilmente (tanto para el propio programador como para otros).
- También incrementa la mantenibilidad, ya que un código claro y ordenado es más sencillo de modificar o ampliar.
- Además, mejora la calidad del diseño del software, ayudando a eliminar dependencias innecesarias y estructuras confusas (code smells), que dificultan el trabajo en equipo.

REFACTORIZACION

1.1 LIMITACIONES DE LA REFACTORIZACIÓN:

La refactorización, aunque es una práctica muy útil para mejorar la calidad del código, presenta una serie de limitaciones que deben tenerse en cuenta antes de aplicarla.

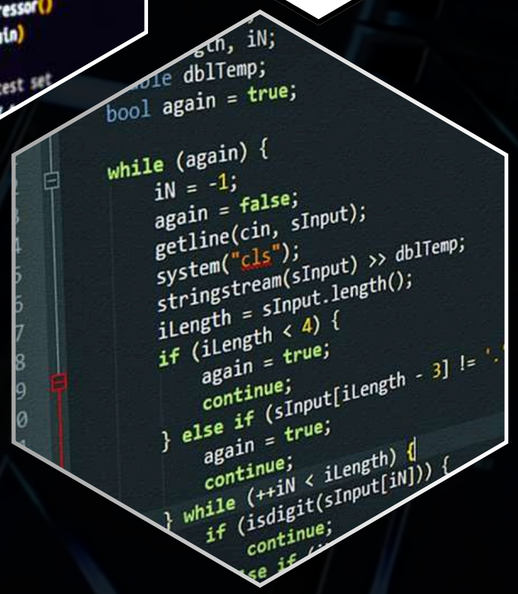
NO AÑADE FUNCIONALIDADES NUEVAS:

En primer lugar, es importante destacar que la refactorización no añade nuevas funcionalidades al software.

Su objetivo no es ampliar lo que el programa hace, sino *mejorar cómo está escrito internamente*.

Por tanto, no debe confundirse con tareas de desarrollo o evolución del sistema, ya que no responde directamente a nuevas necesidades del usuario.

¡Únicamente mejora y optimiza lo que ya esta hecho!



RIESGOS SI NO EXISTEN PRUEBAS:

Otra limitación importante es el riesgo de introducir errores cuando no existen pruebas adecuadas. Si el software *no cuenta con pruebas automatizadas que verifiquen su funcionamiento*, cualquier cambio en el código puede modificar el comportamiento del programa sin que el desarrollador se dé cuenta.

Esto puede provocar fallos difíciles de detectar, especialmente en sistemas grandes o complejos, donde un pequeño cambio puede afectar a muchas partes del código.

REFACTORIZACION

1.2 LIMITACIONES DE LA REFACTORIZACIÓN:

COSTES DE TIEMPO Y DE RECURSOS:

La refactorización también supone un coste de tiempo y recursos.

Aunque a largo plazo mejora la mantenibilidad del software, a corto plazo requiere dedicar demasiadas horas de trabajo que podrían haberse empleado en desarrollar nuevas funcionalidades.

En proyectos con plazos ajustados o presupuestos limitados, esta inversión puede resultar difícil de justificar, lo que hace que muchas veces se posponga o se realice de forma parcial, lo que puede llevar a afectar el mantenimiento de dicho código.

NO SOLUCIONA PROBLEMAS DE DISEÑO PROFUNDO:

Además, la refactorización no soluciona problemas de diseño profundo.

Si un sistema tiene una arquitectura mal planteada desde su origen, aplicar pequeñas mejoras en el código no será suficiente para corregir los errores estructurales.

En estos casos, sería necesario realizar un rediseño completo del sistema, ya que la refactorización solo actúa a nivel interno del código, no sobre decisiones arquitectónicas fundamentales.

NECESIDAD DE EXPERIENCIA Y CONOCIMIENTO DEL CODIGO:

Por último, la refactorización requiere experiencia y conocimiento del código.

Si el desarrollador no entiende bien el funcionamiento del sistema, puede aplicar cambios incorrectos que empeoren la situación en lugar de mejorarla.

Por ello, es fundamental que la refactorización se realice de forma planificada, con apoyo de pruebas y por personas que conozcan bien el proyecto en cuestión.

REFACTORIZACION

PATRONES DE REFACTORIZACIÓN MAS USUALES:

Los patrones de refactorización son técnicas habituales que se aplican para mejorar la estructura interna del código sin modificar su comportamiento, además existen varios patrones comunes que se aplican con frecuencia. Entre ellos se encuentra:

RENOMBRAR VARIABLES/MÉTODOS:

Este patrón consiste en cambiar nombres poco claros, ambiguos o genéricos por otros que describan mejor su función. Un nombre adecuado facilita enormemente la comprensión del código y reduce la necesidad de comentarios adicionales.

MOVER MÉTODO/CLASE:

Se utiliza cuando una función o una clase no está ubicada en el lugar más adecuado del programa. A medida que un proyecto crece, es común que ciertas responsabilidades queden mal distribuidas. Mediante este patrón, los métodos o clases se trasladan a donde tienen más sentido desde el punto de vista del diseño, mejorando la cohesión y reduciendo dependencias innecesarias entre partes del sistema.

SIMPLIFICACION DE CONDICIONALES:

Consiste en reducir la complejidad de estructuras condicionales muy largas o difíciles de entender. Esto puede implicar reorganizar las condiciones, eliminar casos innecesarios o dividir la lógica en métodos más pequeños y claros. El objetivo es que el flujo del programa sea más fácil de seguir y que el código resulte más limpio y comprensible.

EXTRAER MÉTODO:

Se utiliza cuando un método es demasiado largo o realiza varias tareas diferentes. En este caso, una parte del código se extrae y se convierte en un nuevo método con una responsabilidad concreta. Esto mejora la legibilidad, permite reutilizar código y facilita las pruebas.

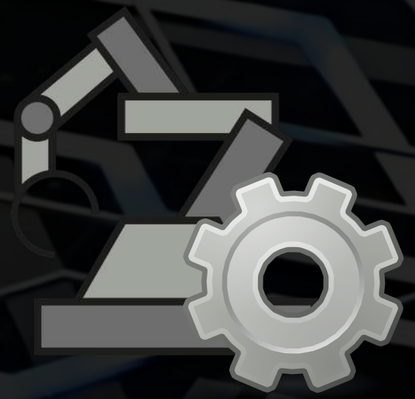
ELIMINAR CÓDIGO DUPLICADO:

Cuando el mismo fragmento de código aparece repetido en diferentes partes del programa, se extrae y se centraliza en un único método o clase. Esto facilita el mantenimiento, ya que cualquier cambio solo debe realizarse una vez, y reduce la probabilidad de errores causados por modificaciones inconsistentes.

REFACTORIZACION

REFACTORIZACION Y PRUEBAS:

Las pruebas de refactorización son conjuntos de pruebas automáticas, principalmente unitarias y de integración, que se ejecutan antes y después de reestructurar el código fuente para asegurar que su comportamiento no cambie.

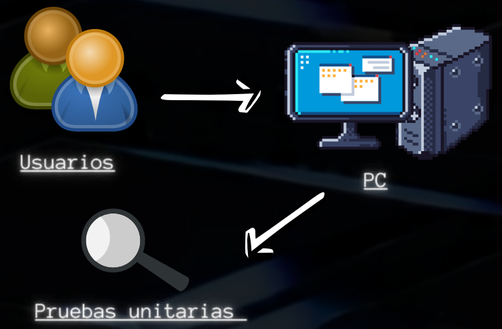


IMPORTANCIA DE LAS PRUEBAS AUTOMATIZADAS:

La automatización de las pruebas permite ejecutarlas de forma rápida y repetida cada vez que se realiza un cambio en el código. Esto resulta especialmente útil en procesos de refactorización, donde se realizan muchas modificaciones pequeñas y continuas. Al ejecutar las pruebas automáticamente, se reduce el riesgo de errores humanos y se gana confianza en los cambios realizados.

EVITAR QUE EL COMPORTAMIENTO CAMBIE:

Antes de iniciar un proceso de refactorización, es recomendable contar con un conjunto de pruebas que cubran las partes más importantes del programa. Estas pruebas actúan como una red de seguridad, ya que permiten detectar de forma inmediata cualquier fallo provocado por los cambios realizados. Durante la refactorización, las pruebas deben ejecutarse con frecuencia para asegurarse de que cada modificación mantiene el comportamiento original del software.



PRUEBAS UNITARIAS COMO APOYO:

Las pruebas unitarias facilitan la mejora del código, ya que permiten refactorizar con mayor seguridad. Cuando el desarrollador sabe que las pruebas cubrirán los posibles fallos, puede centrarse en mejorar la estructura del código sin miedo a romper su funcionamiento. Por este motivo, la refactorización y las pruebas automatizadas deben considerarse prácticas complementarias e inseparables.

REFACTORIZACION

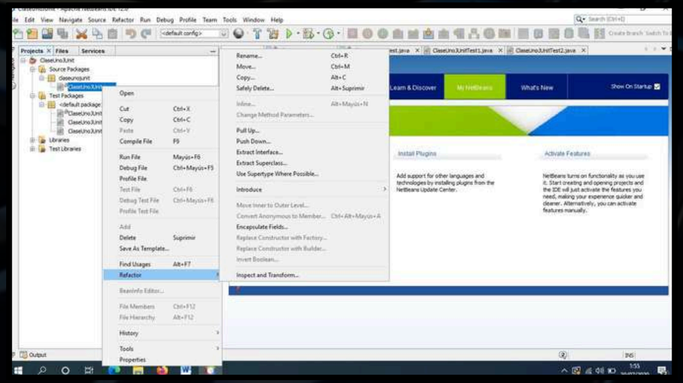
1.1 HERRAMIENTAS DE AYUDA A LA REFACTORIZACIÓN:

Los entornos de desarrollo integrados, conocidos como IDE, ofrecen numerosas herramientas que facilitan el proceso de refactorización. Estas herramientas permiten realizar cambios de forma automática y segura, reduciendo el riesgo de errores. Por ejemplo, al renombrar una variable o un método, el IDE se encarga de actualizar todas las referencias en el proyecto, evitando fallos por olvidos o cambios incompletos.

NETBEANS:

En NetBeans, las funciones de refactorización (como cambiar el renombrado de variables, métodos y clases) se activan desde el menú de refactorización o mediante atajos de teclado. Por ejemplo, al seleccionar una variable y elegir la opción de renombrar, NetBeans no solo cambia el nombre en esa línea, sino también en todo el proyecto. Estas funciones vienen activadas por defecto y no requieren ninguna configuración previa, ya que están integradas en el IDE desde su instalación.

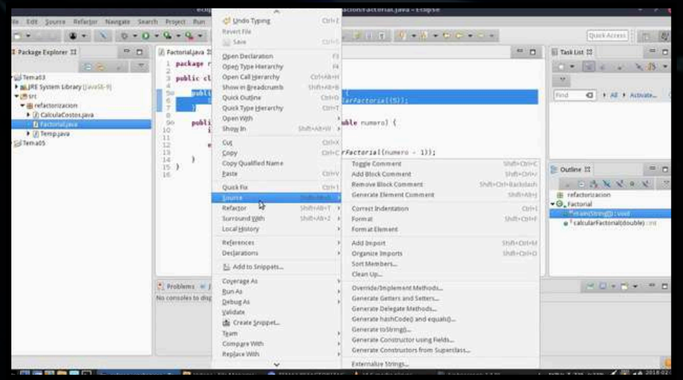
NETBEANS



ECLIPSE:

Por su parte, Eclipse cuenta con herramientas de refactorización muy completas que se integran directamente en el editor. Las herramientas de refactorización están incluidas de forma nativa y se accede a ellas desde el menú contextual del editor o desde el menú principal. Estas permiten mover métodos o clases de un paquete a otro manteniendo intacto el funcionamiento del programa. También facilita la eliminación de código duplicado, ya que puede detectar fragmentos repetidos y sugerir su refactorización en un único método reutilizable.

ECLIPSE



REFACTORIZACION

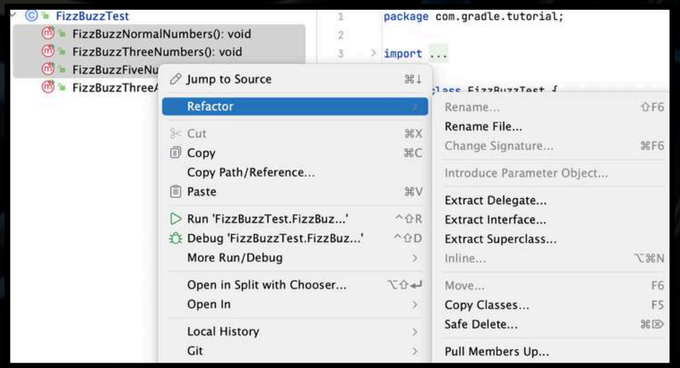
1.2 HERRAMIENTAS DE AYUDA A LA REFACTORIZACIÓN:

Los entornos de desarrollo integrados, conocidos como IDE, ofrecen numerosas herramientas que facilitan el proceso de refactorización. Estas herramientas permiten realizar cambios de forma automática y segura, reduciendo el riesgo de errores. Por ejemplo, al renombrar una variable o un método, el IDE se encarga de actualizar todas las referencias en el proyecto, evitando fallos por olvidos o cambios incompletos.

INTELLIJ:

En IntelliJ, las funciones automáticas de refactorización están especialmente orientadas a mejorar la calidad del diseño del código. Este IDE permite identificar métodos demasiado largos o clases con demasiadas responsabilidades, y ofrece opciones para extraer métodos o reorganizar el código de manera más clara. A su vez, IntelliJ analiza el impacto de cada cambio antes de aplicarlo, mostrando al desarrollador qué partes del proyecto se verán afectadas, lo que aumenta la seguridad del proceso de refactorización.

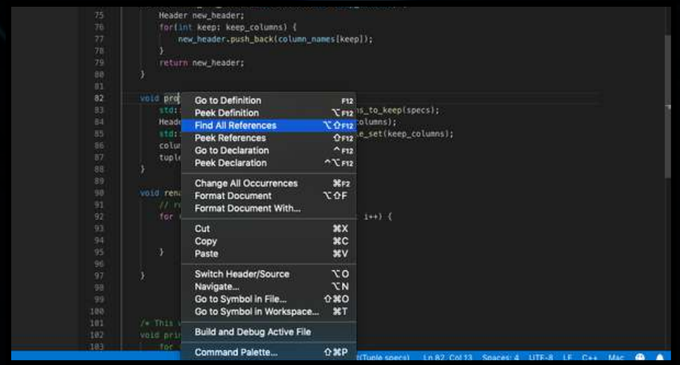
INTELLIJ



VISUAL STUDIO CODE:

Finalmente, Visual Studio Code, aunque es un entorno más ligero, también dispone de herramientas automáticas de refactorización gracias a sus extensiones y al soporte de los lenguajes de programación. Visual Studio Code permite renombrar variables y métodos de forma global dentro del proyecto, así como reorganizar código y simplificar estructuras. Estas funciones ayudan a mantener el código limpio y ordenado incluso en proyectos colaborativos.

VISUAL STUDIO CODE



CONTROL DE VERSIONES

1.1 DESARROLLOS COLECTIVOS:

En el desarrollo de software es muy habitual que varias personas trabajen al mismo tiempo sobre un mismo proyecto. A este tipo de trabajo se le denomina desarrollo colectivo.

¿POR QUE EL DESARROLLO COLECTIVO ES ESENCIAL?:

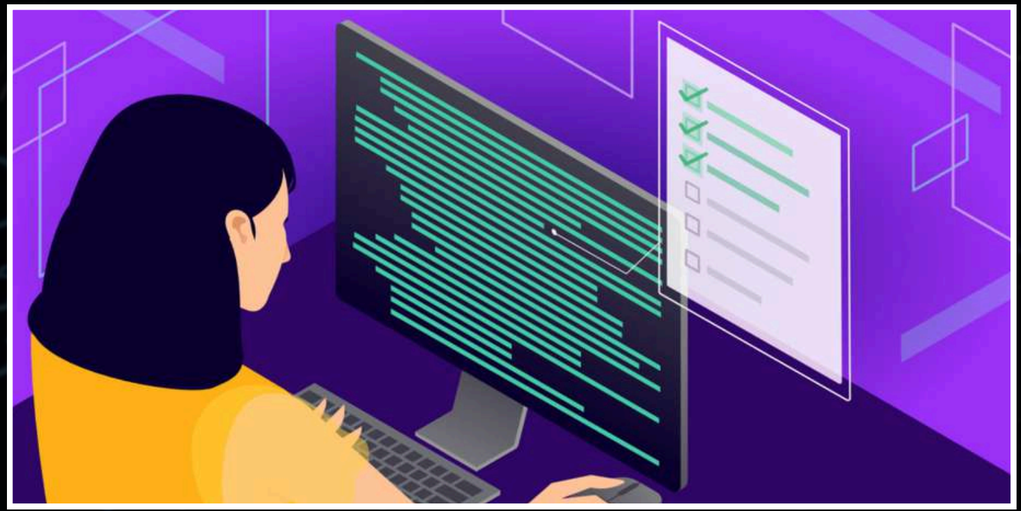
Cada programador puede estar encargado de una parte distinta del sistema, pero todos contribuyen al mismo código fuente. Sin un sistema adecuado, este tipo de trabajo sería caótico, ya que los cambios de una persona podrían sobrescribir los de otra. Por este motivo, es necesario utilizar herramientas que permitan coordinar el trabajo en equipo y mantener el control sobre el proyecto.

CONTROL DE VERSIONES

1.2 DESARROLLOS COLECTIVOS:

CONTROL DE CAMBIOS:

El control de versiones permite gestionar todos los cambios que se realizan en un proyecto a lo largo del tiempo. Cada modificación queda registrada, indicando qué se ha cambiado, quién lo ha hecho y cuándo. Esto es especialmente importante en proyectos colaborativos, ya que permite revisar el trabajo de los distintos miembros del equipo y entender la evolución del software. Además, facilita detectar errores y volver a versiones anteriores si algo no funciona correctamente.



SEGUIMIENTO DE AUTORES:

Una de las grandes ventajas del control de versiones es que permite realizar un seguimiento detallado de los autores y de las modificaciones realizadas. Cada cambio queda asociado a un desarrollador concreto, lo que ayuda a identificar responsabilidades y a resolver dudas sobre el código. Este seguimiento mejora la organización del equipo y fomenta buenas prácticas, ya que todos los cambios quedan documentados de forma automática.

CONTROL DE VERSIONES

HERRAMIENTAS DE CONTROL DE VERSIONES:

Existen diversas herramientas diseñadas específicamente para el control de versiones. Entre las más utilizadas se encuentran Git, SVN y Mercurial.

SVN:

SVN, también conocido como Subversion, es un sistema de control de versiones centralizado. En este modelo, existe un único repositorio central al que se conectan todos los desarrolladores. Cada persona descarga una copia del proyecto, pero los cambios dependen siempre del servidor central. SVN es más sencillo de entender para principiantes y ha sido muy utilizado en proyectos tradicionales, aunque actualmente se usa menos que Git. Aun así, sigue siendo una opción válida en ciertos entornos empresariales.

GIT:

Git es actualmente la herramienta de control de versiones más utilizada. Se trata de un sistema distribuido, lo que significa que cada desarrollador tiene una copia completa del repositorio en su ordenador. Esto permite trabajar sin conexión a internet y realizar cambios de forma local. A su vez, destaca por su rapidez, su flexibilidad y su potente sistema de ramas, que facilita el desarrollo paralelo de nuevas funcionalidades. En proyectos reales, Git suele utilizarse junto con plataformas como GitHub, GitLab o Bitbucket, que permiten compartir el repositorio y colaborar de forma remota.

MERCURIAL:

Mercurial es otro sistema de control de versiones distribuido, similar a Git en su funcionamiento básico. Al igual que Git, permite trabajar con copias locales completas del proyecto y realizar cambios sin necesidad de conexión constante. Mercurial se caracteriza por ser más sencillo de usar y por ofrecer una interfaz más clara para algunos usuarios. Aunque no es tan popular como Git, se ha utilizado en proyectos reales donde se valora la simplicidad y la estabilidad.

En conjunto, Git, SVN y Mercurial son herramientas fundamentales en el desarrollo de software, ya que permiten controlar versiones, coordinar el trabajo en equipo y mantener un historial completo de los cambios realizados en un proyecto.

CONTROL DE VERSIONES

CARACTERÍSTICAS PRINCIPALES DE CONTROL DE VERSIONES:

HISTORIAL DE VERSIONES::

Una de las características principales de los sistemas de control de versiones es el *historial de versiones*. Esta característica permite almacenar y consultar todas las versiones que ha tenido un proyecto desde su inicio. Gracias al historial, es posible ver qué cambios se han realizado en cada momento, quién los ha hecho y por qué. Esto resulta muy útil para entender la evolución del software y para volver a una versión anterior si aparece algún error importante.

FUSIONES (MERGE):

La fusión de cambios, conocida como merge, es otra característica clave del control de versiones. Una vez que el trabajo realizado en una rama ha finalizado y ha sido comprobado, se fusiona con la rama principal del proyecto. Este proceso integra los cambios realizados y permite que formen parte del proyecto común. La fusión es esencial para mantener el trabajo del equipo unido y actualizado.

TRABAJO EN RAMAS:

Otra característica fundamental es el trabajo con ramas, también conocidas como branches. Las ramas permiten desarrollar nuevas funcionalidades, corregir errores o realizar pruebas sin afectar al código principal del proyecto. Cada desarrollador puede trabajar en su propia rama de forma independiente, lo que facilita el trabajo en paralelo y reduce el riesgo de errores en la versión principal del software.

RESOLUCIÓN DE CONFLICTOS:

Por último, los sistemas de control de versiones incluyen mecanismos para la resolución de conflictos. Los conflictos aparecen cuando dos o más desarrolladores modifican la misma parte de un archivo de forma incompatible. En estos casos, el sistema avisa del problema y permite al desarrollador decidir qué cambios se mantienen. Esta característica es imprescindible para garantizar que el código final sea coherente y funcione correctamente.

CONTROL DE VERSIONES

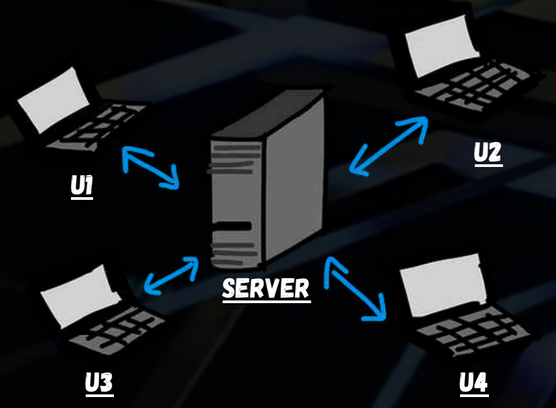
ESTRUCTURA:

Los sistemas de control de versiones tienen una estructura organizada que permite coordinar el trabajo de varios desarrolladores de manera eficiente.Existen varios tipos de estructuras:

MODELO CLIENTE/SERVIDOR

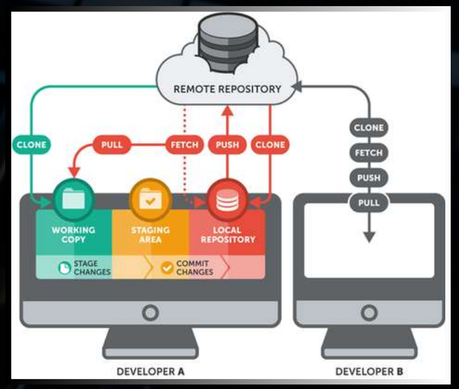
En primer lugar, muchos sistemas utilizan un modelo cliente/servidor, donde existe un servidor central que almacena la versión oficial del proyecto y varios clientes que son las copias locales de los desarrolladores.

Cada programador trabaja en su cliente y se comunica con el servidor para actualizar y compartir cambios. Este modelo asegura que siempre exista un punto central de referencia, evitando la pérdida de información y manteniendo la coherencia del proyecto.



REPOSITORIO REMOTO:

El repositorio remoto es el lugar donde se guarda la versión central del proyecto. Este repositorio contiene todo el historial de cambios, las ramas de desarrollo y los archivos principales. Todos los desarrolladores deben conectarse al repositorio remoto para descargar el proyecto inicialmente, subir sus cambios o actualizar su copia local con las modificaciones de otros miembros del equipo. El repositorio remoto es esencial para que los equipos puedan colaborar sin conflictos y mantener un control sobre el desarrollo.



COPIAS LOCALES:

Finalmente, cada desarrollador trabaja con una copia local del proyecto en su propio ordenador. Esta copia permite modificar el código, probar nuevas funcionalidades y realizar experimentos sin afectar la versión oficial en el repositorio remoto. Cuando se completan los cambios, el desarrollador sincroniza su copia local con el repositorio remoto, subiendo sus modificaciones y descargando los cambios realizados por otros compañeros. De esta manera, el trabajo de todos se integra de forma ordenada y segura.

CONTROL DE VERSIONES

1.1 OPERACIONES CON CLIENTES DE CONTROL DE VERSIONES:

El uso de un sistema de control de versiones implica realizar una serie de operaciones básicas.

DESCARGA INICIAL DEL PROYECTO:

La primera operación que realiza un desarrollador en un sistema de control de versiones es la descarga inicial del proyecto. Esto consiste en obtener una copia completa del proyecto desde el repositorio remoto para poder trabajar sobre él en el ordenador local. Esta copia incluye todos los archivos, carpetas y el historial de cambios, y permite que el desarrollador tenga un punto de partida para comenzar a modificar el código sin alterar la versión principal.

ACTUALIZACIÓN DE FICHEROS EN LOCAL:

Después de realizar cambios, se realiza la actualización de ficheros en local, que consiste en sincronizar la copia del desarrollador con los cambios que otros miembros del equipo han subido al repositorio remoto. Esta operación asegura que todos los desarrolladores trabajen sobre la versión más reciente del proyecto y evita conflictos por modificaciones desactualizadas.

MODIFICACIONES DE FICHEROS:

Una vez descargado el proyecto, el siguiente paso es la modificación de ficheros. El desarrollador realiza los cambios necesarios en su copia local del proyecto, ya sea para añadir nuevas funcionalidades, corregir errores o mejorar el código existente. Estas modificaciones se hacen de manera independiente y no afectan inmediatamente al repositorio remoto, lo que permite trabajar de forma segura.

SUBIDA DE CAMBIOS AL REPOSITORIO:

La subida de cambios al repositorio es la operación mediante la cual los cambios realizados en la copia local se envían al repositorio remoto. Esto permite que las modificaciones estén disponibles para todos los miembros del equipo y formen parte del proyecto oficial. Antes de subir los cambios, los sistemas de control de versiones suelen verificar que no existan conflictos con modificaciones de otros desarrolladores.

CONTROL DE VERSIONES

1.2 OPERACIONES CON CLIENTES DE CONTROL DE VERSIONES:

El uso de un sistema de control de versiones implica realizar una serie de operaciones básicas.

COMPARACIÓN ENTRE VERSIONES:

Otra operación importante es la comparación entre versiones, que permite ver qué cambios se han realizado entre distintas versiones del proyecto. Esta funcionalidad ayuda a entender la evolución del código, detectar errores introducidos en versiones recientes y analizar cómo se ha modificado cada archivo a lo largo del tiempo.

RESOLUCIÓN DE CONFLICTOS:

La resolución de conflictos es necesaria cuando dos o más desarrolladores han modificado la misma parte de un archivo y los cambios son incompatibles. El sistema de control de versiones alerta sobre el conflicto y permite decidir cuál de los cambios se mantiene o cómo combinarlos, asegurando que el código final sea coherente.

RESTAURACIÓN DE VERSIONES ANTERIORES:

La restauración de versiones anteriores permite volver a un estado previo del proyecto si algún cambio provoca errores o problemas importantes. Gracias a esta función, es posible deshacer modificaciones de manera segura sin perder el resto del trabajo realizado, lo que protege la estabilidad del software.

CONSULTA DEL HISTORIAL:

Por último, la consulta del historial permite revisar todos los cambios realizados en el proyecto, quién los hizo y cuándo. Esta operación es fundamental para documentar el trabajo del equipo, entender la evolución del software y facilitar la identificación de errores o modificaciones problemáticas.

DOCUMENTACION DEL SOFTWARE:

USOS DEL COMENTARIOS:

USO PRINCIPAL:

Los comentarios en el código se utilizan para explicar partes importantes o complejas del programa. Su finalidad es ayudar a otros desarrolladores, o al propio autor en el futuro, a entender la intención del código. Sin embargo, es importante seguir buenas prácticas y no abusar de los comentarios. Un buen código debe ser claro por sí mismo, y los comentarios deben aportar información útil, no repetir lo que el código ya muestra de forma evidente.

BENEFICIOS DE LOS COMENTARIOS:

Cuando un programador o un equipo revisa el código, los comentarios permiten entender rápidamente qué hace cada sección, cómo funciona un algoritmo o cuál es la finalidad de ciertas instrucciones, evitando tener que deducirlo únicamente leyendo el código.

Otro beneficio importante es que los comentarios facilitan el mantenimiento del software. Cuando se necesita corregir errores o actualizar funciones, los comentarios ayudan a identificar rápidamente qué partes del código están relacionadas con una funcionalidad concreta, reduciendo el riesgo de modificar algo que no se debía o introducir nuevos errores.

Los comentarios también contribuyen a mejorar la colaboración en equipo. En proyectos donde varios desarrolladores trabajan sobre el mismo código, los comentarios permiten transmitir información sobre la lógica de programación, decisiones de diseño o advertencias sobre posibles problemas.

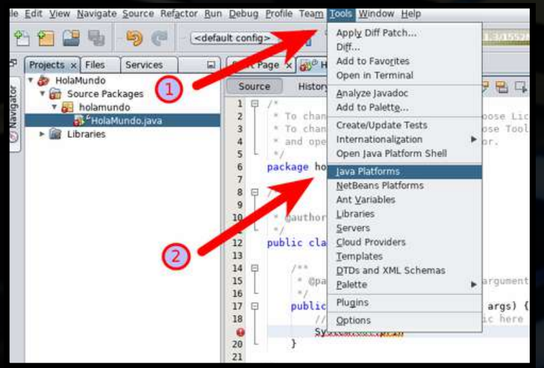
DOCUMENTACION DEL SOFTWARE:

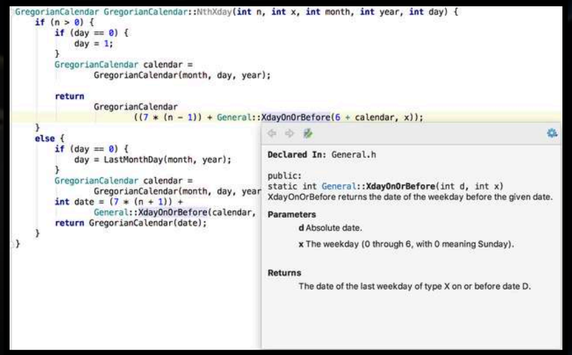
HERRAMIENTAS DE DOCUMENTACIÓN AUTOMÁTICA:

Existen herramientas que permiten generar documentación automáticamente a partir del propio código fuente. Estas herramientas utilizan comentarios especiales para crear documentos estructurados que describen clases, métodos y funciones.

JAVADOC (JAVA):

Es una herramienta utilizada principalmente en el lenguaje de programación Java. Su funcionamiento se basa en la escritura de comentarios especiales dentro del código, normalmente colocados encima de clases, métodos y atributos. Estos comentarios describen qué hace cada elemento del programa, qué parámetros recibe y qué valores devuelve. Una vez escritos, JavaDoc analiza el código y genera automáticamente documentación en formato HTML, que puede consultarse fácilmente desde un navegador.



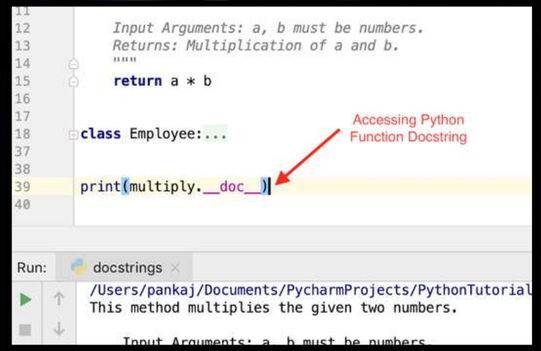


DOXYGEN (C Y C++):

Es una herramienta de documentación automática más general, ya que puede utilizarse con varios lenguajes de programación como C, C++, Java o Python. Al igual que JavaDoc, utiliza comentarios especiales para extraer información del código. Doxygen genera documentación estructurada que puede incluir descripciones, diagramas y relaciones entre clases, lo que resulta muy útil en proyectos grandes o complejos.

DOCSTRINGS (PYTHON):

Son una forma de documentación integrada en algunos lenguajes de programación, como Python. Consisten en cadenas de texto colocadas al inicio de funciones, métodos o clases, que explican su funcionamiento. Estas cadenas pueden ser leídas directamente desde el propio lenguaje o utilizadas por herramientas externas para generar documentación automática. Los docstrings facilitan que la documentación esté siempre ligada al código y sea fácil de mantener.



En conjunto, JavaDoc, Doxygen y los docstrings permiten crear documentación clara, actualizada y accesible, mejorando la comprensión del software y facilitando su mantenimiento.

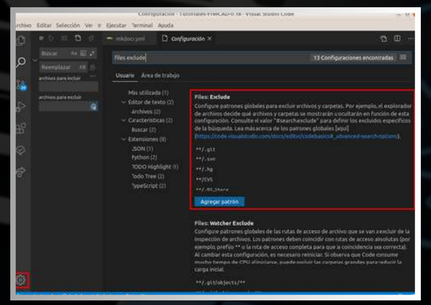
DOCUMENTACION DEL SOFTWARE:

ALTERNATIVAS DE DOCUMENTACIÓN:

Además de la documentación generada automáticamente a partir del código, existen otras alternativas de documentación que son muy utilizadas en proyectos de software. Estas alternativas permiten explicar aspectos del proyecto que no siempre se reflejan directamente en el código, como el uso del programa, su estructura general o las decisiones de diseño tomadas durante el desarrollo.

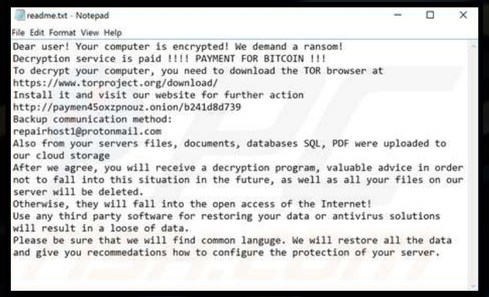
WIKIS DEL PROYECTO:

Una de las alternativas más comunes son las wikis de proyecto. Las wikis permiten crear documentación colaborativa, donde varios miembros del equipo pueden añadir, modificar o actualizar información. En ellas se suele incluir información sobre la instalación del software, su funcionamiento, guías de uso y procedimientos internos. Al ser fáciles de editar, las wikis se adaptan bien a proyectos que evolucionan con el tiempo.



DIAGRAMAS UML:

Por último, los diagramas UML se utilizan para representar gráficamente la estructura y el comportamiento del sistema. A través de diagramas de clases, de secuencia o de casos de uso, los diagramas UML permiten visualizar cómo se organizan los componentes del software y cómo interactúan entre sí. Esta representación gráfica facilita la comprensión del sistema y complementa la documentación escrita.

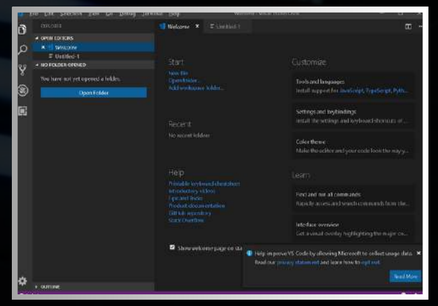


ARCHIVOS README:

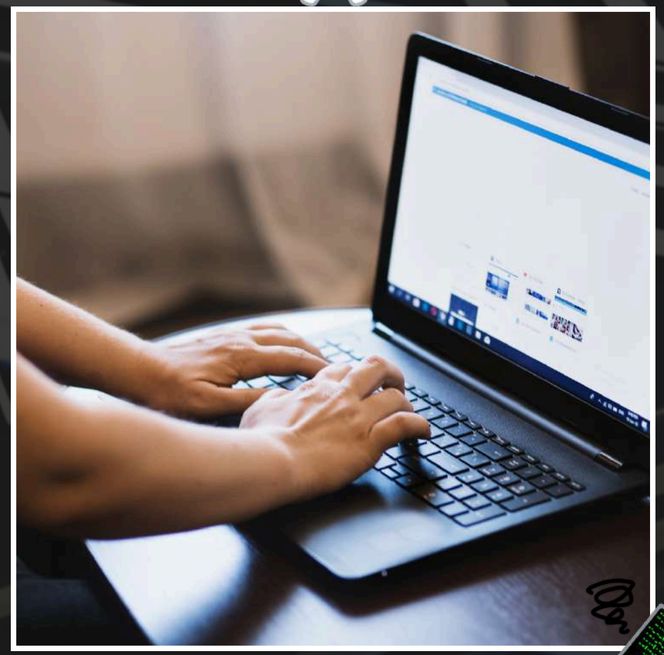
Otra alternativa muy utilizada es el archivo README. Este archivo suele ser el primer documento que se consulta al acceder a un proyecto y contiene información básica pero esencial, como la descripción del software, los requisitos necesarios, las instrucciones de instalación y una explicación general de su uso. El README permite entender rápidamente de qué trata el proyecto sin necesidad de leer todo el código.

MANUALES TECNICOS:

Los manuales técnicos constituyen otra forma importante de documentación. Estos documentos están orientados a desarrolladores o personal técnico y explican en detalle el funcionamiento interno del software, su arquitectura, las tecnologías utilizadas y los procedimientos de mantenimiento. Los manuales técnicos son especialmente útiles en proyectos grandes o cuando el software debe mantenerse durante largos periodos de tiempo.



CONCLUSIÓN DEL PROYECTO:



A lo largo de este proyecto se han analizado tres aspectos fundamentales del desarrollo de software: la refactorización del código, el control de versiones y la documentación del software. Estos tres elementos no solo forman parte de las buenas prácticas de programación, sino que son pilares esenciales para garantizar la calidad, la estabilidad y la evolución de cualquier proyecto informático.

La refactorización permite mejorar la estructura interna del código sin alterar su comportamiento, haciendo que los programas sean más legibles, mantenibles y fáciles de entender. Gracias a ella, el software puede adaptarse mejor a futuros cambios, reducir errores y facilitar el trabajo tanto individual como en equipo. Sin embargo, este proceso debe realizarse con cuidado, apoyándose en pruebas y herramientas adecuadas, para evitar riesgos y asegurar que el funcionamiento del programa se mantiene intacto.

El control de versiones, por su parte, se presenta como una herramienta imprescindible en el desarrollo colaborativo. Sistemas como Git, SVN o Mercurial permiten gestionar los cambios del proyecto, coordinar el trabajo de varios desarrolladores y mantener un historial completo de las modificaciones realizadas. Gracias al uso de repositorios, ramas y mecanismos de resolución de conflictos, el control de versiones garantiza que el trabajo en equipo sea ordenado, seguro y eficiente.

La documentación del software completa este proceso, asegurando que el conocimiento sobre el proyecto no se pierda con el tiempo. Mediante comentarios bien utilizados, herramientas de documentación automática y alternativas como README, wikis, manuales técnicos y diagramas UML, se consigue que el software sea comprensible y fácil de mantener. Una buena documentación facilita la incorporación de nuevos desarrolladores, reduce errores y asegura la continuidad del proyecto a largo plazo.

En conjunto, todas estos aspectos no deben verse como tareas secundarias, sino como prácticas esenciales del desarrollo profesional del software. Su correcta aplicación mejora la calidad del código, fortalece el trabajo en equipo y garantiza que los proyectos puedan crecer, mantenerse y evolucionar de forma sólida y organizada.

Este proyecto demuestra que un buen software no solo se mide por lo que hace, sino por cómo está construido y cómo puede sostenerse en el tiempo.

¡MUCHAS GRACIAS POR VER!